VolleyLive — Product Requirements Document (PRD)

Product	VolleyLive: volleyball live scoring, teams, tournaments and nearby games
Platforms	Android and iOS (one codebase). A web preview exists for development only.
Tech stack	React Native, Expo SDK 57, TypeScript, Expo Router; Firebase (Phone Auth, Firestore, Cloud Storage, Cloud Functions); Expo Push (FCM/APNs)
Owner	Akshit Kumar
Document version	1.0 (draft for review)
Date	10 October 2026
Status	v1 scope agreed. Implementation status was checked against the codebase at `/workspace/volleylive` (see Appendix A).

> **How to read this document.** Every requirement has a stable ID (for example `TEAM-03`) and testable acceptance criteria. A PRD Reviewer will check the build against these IDs. Each requirement carries a status tag based on a code review on 10 Oct 2026:
>
> - **[Built]**: implemented in code and consistent with the requirement.
> - **[Partial]**: some of it is implemented; the gap is described.
> - **[Not built]**: no implementation found.
> - **[Deferred]**: deliberately out of v1.
>
> Numeric targets in this document are **proposed**. None of them are measured values. No usage data exists yet.

---

1. Overview and problem

1.1 Summary

VolleyLive is a cross-platform mobile app for recreational and club volleyball. A single assigned scorer records a match rally by rally on a split-screen tap interface. Everyone else follows the scoreboard in real time. The app also holds teams, player profiles and career stats, match photo albums, team-vs-team challenges, knockout and league tournaments, and a map of nearby games and courts.

1.2 Problem

- **Scoring is manual and lost.** Pick-up and club matches are usually scored on paper, on whiteboards or from memory. Results, set scores and individual contributions are rarely recorded, so players have no history or stats.
- **Followers can't see the score.** Teammates, friends and family who are not at the court have no live view of the score.
- **Organising is fragmented.** Teams, challenges and small tournaments are arranged over chat groups. Fixtures, brackets and tables are drawn up by hand.

- **Games are hard to find.** Players who travel or move to a new city have no easy way to find local games, courts or tournaments.
- **Connectivity is unreliable.** Courts (especially outdoor and beach courts) often have weak mobile data, and online-only scoring tools fail there.

1.3 Product vision

Make scoring any volleyball match fast and reliable, make that score visible live to anyone who cares, and turn every match into lasting stats, history and community.

---

2. Goals and non-goals

2.1 Goals (v1)

ID	Goal
G1	Let one person score a full match with one tap per rally, working offline, without needing to know serve or side-switch rules.
G2	Show followers a live scoreboard that updates within seconds of each rally.
G3	Give every registered player a persistent profile with match history and career totals.
G4	Let captains form real teams (6 starters plus up to 6 subs) using only verified, registered players.
G5	Let organisers run knockout and league tournaments with fixtures, brackets and standings generated automatically.
G6	Help players find games, tournaments and courts near them and ask to join.
G7	Protect players' phone numbers and location data. Score integrity is enforced on the server, not just in the app.

2.2 Non-goals (v1)

- **Live video streaming.** Explicitly out of v1. The data model reserves `Match.stream`. See §4.2.
- Professional and federation features: official FIVB scoresheets, referee or line-judge workflows, libero tracking, rotation or lineup enforcement, substitution tracking during a match.
- Multiple simultaneous scorers or consensus scoring.
- Invite links or SMS invites for unregistered players. These are deliberately excluded: captains can add only numbers that are already registered.
- Payments, court booking, ticketing or prize money.
- In-app chat or social feed.
- Web app for end users. The web build is a developer preview only.
- Advanced analytics such as heat maps or rally-level video tagging.

---

3. Target users and personas

Persona	Description	Key needs
P1 – Scorer / organiser ("Rohan", 28)	Organises weekly games at a community court and usually scores them.	Fast, error-tolerant scoring; undo; works on patchy data; rules handled for them.
P2 – Team captain ("Aarav", 24)	Captains a club or college team.	Build a roster of real players, challenge other teams, schedule matches.

Persona	Description	Key needs
P3 – Player ("Kavya", 21)	Plays regularly and cares about her stats.	Profile, match history, career totals, photos of matches.
P4 – Follower ("Parent / teammate")	Not at the court.	Live score on their phone with no friction.
P5 – Tournament organiser ("Coach Singh", 40)	Runs weekend tournaments of 4–16 teams.	Auto fixtures, brackets and tables; results flow in from scored matches.
P6 – Traveller ("Sara", 30)	Visiting or new to a city.	Find nearby games, courts and tournaments; ask to join a game.

The default market is India: the phone country code defaults to +91, and sample data is set in Delhi. This is a product assumption to confirm (see §14).

---

4. Scope

4.1 v1 (agreed)

1. Phone-number OTP sign-up and profiles (name, photo, position, city).
2. Teams of 6 plus subs. Only registered numbers can be added, through an in-app invite that the player accepts or declines.
3. Split-screen tap scoring with undo, automatic serve tracking and set/match wins, Indoor/Beach/Custom rules, timeouts, side switches, and optional per-player point credit (spike, block, ace, tip).
4. A single assigned scorer per match. Everyone else is read-only.
5. Offline-tolerant scoring (queue and sync).
6. Real-time live scoreboard for followers.
7. Player stats, career totals and match history.
8. Match photo albums.
9. Team challenges with push notifications.
10. Knockout and league tournaments with auto fixtures, brackets and standings.
11. A location on every match, and a nearby games/courts map with "I want to play".

4.2 Later (post-v1 candidates)

Item	Notes
Live video streaming	`Match.stream` (`provider: none/rtmp/webrtc/youtube`, `playbackUrl`, `ingestUrl`) is reserved. The match page shows a "coming in a future version" placeholder.
Server-side geo queries	Geohash or geo-index queries to replace client-side filtering.
Organiser approval of join requests	Accept or decline "I want to play" requests, with notifications.
Captain transfer, co-captains	Not in v1.
Lineups, rotations, substitutions, libero	Not in v1.
Account deletion and data export	Needed for store compliance. Proposed as a v1.x priority (see §13 Risks).
Content moderation and reporting for photos	Proposed for v1.x.

Item	Notes
Multi-country phone support UI	The code supports a default country code. A picker is not built.
Native phone auth (Play Integrity / silent APNs)	Optional switch to `@react-native-firebase/auth`.

---

5. Functional requirements

Conventions: *must* means required for v1 sign-off. Acceptance criteria (AC) use Given/When/Then.

5.1 Authentication and profiles (AUTH, PROF)

AUTH-01 Phone OTP sign-in [Built]

Users sign up and sign in with a mobile number and a 6-digit SMS OTP using Firebase Phone Auth. Numbers are normalised to E.164, with +91 as the default country code.

- AC1: Given a valid number, when the user taps "Send OTP", then an SMS code is requested (after the reCAPTCHA check) and the OTP screen shows the number in E.164 form.
- AC2: Given a correct code, when submitted, then the user is signed in and the session persists across app restarts.
- AC3: Given an invalid number (wrong length or characters), when submitted, then a validation error is shown and no SMS is requested.
- AC4: Given a wrong code, when submitted, then an error is shown and the user stays signed out.

AUTH-02 reCAPTCHA verification inside Expo [Built]

Phone auth must work in Expo Go, development builds and store builds without the native Firebase SDK. It does this with a WebView reCAPTCHA served from the Firebase `authDomain`; the web build uses `RecaptchaVerifier`.

- AC1: Given a store or dev build on Android or iOS, when the user requests an OTP, then the invisible reCAPTCHA completes, or an image challenge appears, and the SMS is sent.

AUTH-03 Demo mode [Built]

When no Firebase config is present, the app runs in Demo mode with local sample data, and any number signs in with code 123456. A banner indicates Demo mode.

- AC1: Given no `EXPO_PUBLIC_FIREBASE_*` env values, when the app starts, then a Demo banner shows on Login and Home, and OTP 123456 signs in.
- AC2: Production builds must have Firebase configured. A release checklist item must confirm that Demo mode is not reachable in store builds. *(Proposed. Not enforced in code.)*

AUTH-04 Sign out [Built]

- AC1: When the user taps "Sign out" on Profile, then the session ends and the Login screen shows.

PROF-01 Profile setup on first sign-in [Built]

A first-time user must create a profile with name (required, 1–60 characters), optional photo, preferred position (Setter, Outside hitter, Middle blocker, Opposite, Libero, All-rounder) and home city.

- AC1: Given a newly verified user with no profile, when sign-in completes, then the "Create your profile" screen shows, and the user cannot reach the tabs until the profile is saved.

- AC2: Given an empty name, when the user saves, then saving is blocked.
- AC3: Given a saved profile, then `users/{uid}.phone` equals the verified phone number on the auth token. The security rules enforce this.

PROF-02 Phone registration index [Built]

When a profile is saved, the user claims `phoneIndex/{e164} → {uid}` for their own verified number only. This index is what makes a number "registered".

- AC1: Given user U with verified number N, when U writes `phoneIndex/N`, then the write succeeds. When U writes `phoneIndex/M` for any M ≠ N, then it is denied.
- AC2: Listing `phoneIndex` is denied. Only an exact `get` by number is allowed.

PROF-03 Edit profile and avatar [Built]

- AC1: Given a signed-in user, when they edit their name, position, city or photo and save, then the changes are visible on their profile and team rosters.
- AC2: Avatar uploads must be images under 10 MB stored under `avatars/{uid}/`. Uploads to another user's path are denied.

PROF-04 Phone number privacy [Partial]

Other users' full phone numbers must not be shown in the UI. Pending invites are shown masked (for example `+91 ••••••0011`).

- AC1: Given a team with pending invites, when any user views the team, then invited numbers are masked. *(Built)*
- AC2: Given user A reads user B's profile, then B's full phone number is not returned. **Gap:** `users/{uid}` stores `phone` and is readable by any signed-in user, so the number can be read outside the UI. Proposed fix: move `phone` to a private sub-document or rely on `phoneIndex` alone.

5.2 Teams (TEAM)

TEAM-01 Create team [Built]

Any user can create a team with a name (up to 40 characters), a colour and an optional city. The creator becomes captain and the first member.

- AC1: When a user creates a team, then `captainId = uid`, `memberIds = [uid]` and `pendingPhones = []`. The rules reject any other initial state.

TEAM-02 Roster size: 6 starters plus up to 6 subs [Partial]

A team has up to 6 starters and up to 6 substitutes (12 members in total). The captain can move members between starters and subs.

- AC1: Given 6 starters, when the captain tries to make a sub a starter, then the app refuses with "You already have 6 starters…". *(Built, client-side)*
- AC2: Given 12 members plus pending invites, when the captain adds another number, then the app refuses. *(Built, client-side; the rules cap `memberIds` and `pendingPhones` at 12 each)*
- AC3: Given a team that already has 6 starters, when an invited player accepts, then they are added as a sub. **Gap:** this happens in Demo mode only. The Firebase backend's `acceptTeamInvite` adds the player to `memberIds` without updating `subIds`, so a 7th "starter" can appear.
- AC4: The security rules should enforce a combined limit of 12 for members plus pending invites, and at most 6 subs. **Gap:** not enforced on the server.

TEAM-03 Add players only if they are registered [Built]

The captain adds a player by mobile number. If the number is not in `phoneIndex`, it is refused with the message *"This number isn't on VolleyLive yet. Ask them to register first, then add them again."* Nothing is stored. There are no invite links or SMS invites.

- AC1: Given an unregistered number, when the captain taps "Add by mobile number", then that message shows and the team document is unchanged.
- AC2: Given a registered number, when added, then it is appended to `pendingPhones` and the message "Invite sent to {name}..." shows.
- AC3: A direct Firestore write that adds an unregistered number to `pendingPhones` is denied by the rules, as is adding more than one new number per write.
- AC4: Duplicate invites and numbers of existing members are refused with "Already invited." or "{name} is already in the team."

TEAM-04 Invited player accepts or declines [Built]

Registered invitees see the invite on Home ("{Team} wants you on the team") with **Join team** and **Decline** buttons. A push notification is also sent (NOTIF-01).

- AC1: Given user U whose verified phone is in `pendingPhones`, when U accepts, then U is added to `memberIds` and the number is removed from `pendingPhones`, in one write.
- AC2: When U declines, then only the number is removed.
- AC3: User U cannot accept another user's invite, or add anyone other than themselves while accepting. The rules check this against `request.auth.token.phone_number`.

TEAM-05 Membership integrity: no bypassing the invite [Partial]

A player must never become a member without accepting an invite.

- AC1: Through the UI, players are added only through the accept flow. *(Built)*
- AC2: A captain's direct write that adds an arbitrary uid to `memberIds` is denied. **Gap:** the captain update rule doesn't restrict additions to `memberIds`. It only checks that the captain stays a member and the size is at most 12. Proposed fix: the captain may only *remove* from `memberIds`, and additions may come only through `acceptingInvite()`.

TEAM-06 Remove, leave, cancel invite [Built]

The captain can remove members and cancel pending invites. Non-captain members can leave.

- AC1: When a member taps "Leave this team" and confirms, then they are removed from `memberIds` and `subIds`. The captain cannot use the leave path.
- AC2: When the captain taps the delete icon on a pending invite, then that number is removed.

TEAM-07 Only the captain edits the team [Built]

- AC1: A non-captain's edit (other than accepting or declining their own invite, or leaving) is denied by the rules.

5.3 Live scoring (SCORE)

SCORE-01 Split-screen tap scoring [Built]

The scoring screen shows two large halves, one per team, in team colours. Each half shows the team name, sets won, current points and a serving badge. Tapping a half awards that team the rally (rally scoring). Each tap gives haptic feedback.

- AC1: Given a live match, when the scorer taps Team A's half, then Team A's points go up by 1 within one frame on the device, and the event is appended to the log.
- AC2: Taps after the match has finished are ignored.

SCORE-02 Undo [Built]

- AC1: Given at least one event, when the scorer taps Undo, then the last event (point or timeout) is removed and all derived state (score, serve, sets, side-switch prompts) is recalculated.
- AC2: Given no events, the Undo button is disabled.
- AC3: Undoing the point that won a set or the match restores the previous set or match state.

SCORE-03 Automatic serve tracking [Built]

The rally winner serves next. Service alternates at the start of each set, starting from the configured first server.

- AC1: Given A served and B wins the rally, then the serving badge moves to B.
- AC2: Given set 1 started with A serving, then set 2 starts with B serving.

SCORE-04 Set point, match point, set and match wins [Built]

The engine detects set point and match point (shown as a badge) and set and match completion, according to the rules (§SCORE-05).

- AC1: Indoor rules at 24–23: "SET POINT" shows for the leading team. At 25–23 the set is won. At 25–24 play continues (win by 2).
- AC2: When the required number of sets is won (for example 3 of 5), then status becomes `finished`, `finishedAt` is set and further taps are ignored.

SCORE-05 Rule presets: Indoor, Beach, Custom [Built]

Rule	Indoor	Beach	Custom
Best of	5	3	1/3/5
Points per set	25	21	≥1
Deciding set	15	15	≥1
Win by	2	2	≥1
Point cap	none	none	optional, ≥ points per set
Timeouts per team per set	2	1	≥0
Side switch	at 8 in deciding set	every 7 points (every 5 in deciding set)	configurable

- AC1: Changing any preset value marks the rules as "Custom".
- AC2: Invalid values (best-of not in {1,3,5}, points < 1, win-by < 1, cap < points per set, negative timeouts) are rejected.

SCORE-06 Timeouts [Built]

- AC1: Given a team with timeouts left, when the scorer taps "Timeout (n)", then a timeout event is logged and the count goes down.
- AC2: Given 0 timeouts left, the app shows "No timeouts left" and logs nothing.
- AC3: The timeout allowance resets every set.

SCORE-07 Side switches [Built]

- AC1: When the rules require a side switch (for example 7 combined points in Beach), then a "Switch sides" prompt is shown.
- AC2: The scorer can swap the on-screen halves at any time with "Swap". This is a display change only and doesn't alter the score.

SCORE-08 Optional per-player point credit [Built]

After a point, an optional player strip ("Who scored for {Team}?") lets the scorer credit the point to a player as Spike, Block, Ace or Tip, or skip.

- AC1: Given a point by Team A with a roster, when the scorer picks a player and a kind, then the last point event records `playerId` and `kind`.
- AC2: Credit can only be added to the most recent event, and only if it is a point.
- AC3: Skipping leaves the point uncredited. It still counts toward the team score.
- AC4: Players added only by typed name (guests) can be credited in the match, but these points don't go to any registered profile.

SCORE-09 Match setup [Partial]

Creating a match: pick Team A and Team B (registered team or typed name with optional comma-separated player names), title, rules, first serve, start now or scheduled time, and location.

- AC1: Given the same registered team on both sides, the app shows "Pick two different teams."
- AC2: Given "Start now", the scorer goes straight to the scoring screen and status is `live`.
- AC3: See LOC-01 for the location requirement. **Gap:** location is optional at match creation.

SCORE-10 End or abandon a match [Built]

- AC1: The scorer can mark a match as abandoned from the scoring menu. Status becomes `abandoned` and it no longer appears as live.

SCORE-11 Share result [Built]

- AC1: From the match page, "Share" opens the OS share sheet with a text summary (teams, set scores, location).

5.4 Scorer role and permissions (ROLE)

ROLE-01 Single assigned scorer [Built]

Each match has exactly one `scorerId`. By default this is the creator, or the organiser for tournament fixtures. Only the scorer can change `events`, `summary` and `status`.

- AC1: Given user X ≠ scorerId, when X opens the scoring screen, then the screen shows "Only the assigned scorer can score this match…" with "Watch live".
- AC2: Given user X ≠ scorerId, a direct Firestore update to `events` is denied.
- AC3: The scorer cannot change `createdBy`, and cannot change `scorerId` in a write that also changes the score.

ROLE-02 Hand over scoring [Built]

The creator or the current scorer can reassign the scorer to another match player.

- AC1: When the scorer picks another player chip under "Scorer", then `scorerId` updates and the previous scorer immediately loses write access. The rules test "creator can hand over scoring, then loses score access" covers this.

ROLE-03 Read-only followers [Built]

- AC1: Every signed-in user can view any match page. Only the scorer sees scoring controls.

5.5 Offline-tolerant scoring (OFF)

OFF-01 Local-first scoring [Built]

Every scoring change is first saved on the device (AsyncStorage key `score-cache-{matchId}`, with the full event log and a `pending` flag) and then pushed to Firestore.

- AC1: Given airplane mode, when the scorer records 20 rallies, then all 20 show on the scoring screen and the sync indicator reads "⚠ Offline – saved on phone".
- AC2: Given connectivity returns, then the latest full event log is written, the indicator shows "✓ Synced" and followers see the final state.

OFF-02 Retry and recovery [Partial]

- AC1: Given a failed write that is not a permissions error, the app retries every 5 s with the latest log while the scoring screen is open. *(Built)*
- AC2: Given the app is killed while offline with pending points, when the scorer reopens the match's scoring screen, then the local log is restored if it is newer than the server copy. *(Built)*
- AC3: Pending points must sync without the scorer having to reopen that match's scoring screen, for example through a background sync on app start. **Gap:** retry runs only while the scoring screen is mounted. Firestore uses its default in-memory cache, not persistent IndexedDB/LRU.

OFF-03 Conflict model [Built]

Scoring is event-sourced and there is only one scorer, so the latest full event log from the scorer is the source of truth. Out-of-order writes are dropped with a push sequence counter.

- AC1: Given rapid taps while offline, when sync resumes, then the server log equals the device log exactly.

OFF-04 Followers offline [Not built, by design]

Followers need a connection to receive updates. There is no offline cache of other matches beyond what the Firestore SDK holds in memory.

5.6 Real-time live following (LIVE)

LIVE-01 Real-time scoreboard [Built]

The match page subscribes to `matches/{id}` and shows teams, sets, current points, the serving team, set scores, a LIVE badge, rules, location (with "Directions") and a "You're following this match live" banner.

- AC1: Given a follower on the match page, when the scorer records a point online, then the follower's scoreboard updates without a manual refresh. Proposed target: p95 ≤ 2 s on 4G (see NFR-02).

LIVE-02 Home lists live and upcoming matches [Built]

- AC1: Home shows "Live now" (status `live`) and "Upcoming" (status `scheduled`, sorted by time). The query is limited to 50 matches.

LIVE-03 Match tabs: stats, timeline, photos [Built]

- AC1: The Stats tab shows team totals (points, spikes, blocks, aces, tips, longest run) and per-player points.
- AC2: The Timeline tab shows a point-by-point log per set with running score and credited player.

LIVE-04 Live video [Deferred]

- AC1: The match page shows "Live video streaming is coming in a future version." No streaming is offered in v1.

5.7 Stats and history (STAT)

STAT-01 Per-match stats [Built]

Calculated from the event log: set-by-set scores, team totals per point kind, longest scoring run, and points per player.

- AC1: For a known event log, `playerStats`, `teamTotals` and `longestRun` return the expected values. Covered by Jest tests.

STAT-02 Career totals [Built]

Profile shows matches, wins, losses, win %, points, spikes, blocks, aces and tips, aggregated across matches where the user is a participant.

- AC1: Given a user who has finished 4 matches and won 3, then Win % shows 75%.
- AC2: Given no finished matches, Win % shows "–".
- Note: aggregation runs on the client over up to 200 of the user's matches (query limit). Proposed: server-side aggregates once users pass 200 matches.

STAT-03 Match history [Built]

- AC1: Profile "Match history" lists the user's matches (live, scheduled, finished) and each item opens the match page.

5.8 Match photo albums (PHOTO)

PHOTO-01 Add photos [Built]

Match participants can add photos from the gallery or the camera (camera on native only).

- AC1: Given a participant, when they add a photo, then it uploads to `matches/{matchId}/{uid}/…` and appears in the Photos tab for all viewers in real time.
- AC2: Given a non-participant, the "Add photo" control isn't offered, and the Storage and Firestore rules deny the upload.
- AC3: Non-images and files of 10 MB or more are rejected.

PHOTO-02 View album [Built]

- AC1: The Photos tab shows the count "Photos (n)" and a grid, newest first. With no photos, it shows "No photos yet".

PHOTO-03 Delete own photo [Partial]

- AC1: The rules allow an uploader to delete only their own photo. **Gap:** there is no delete control in the UI or backend.

PHOTO-04 Report or moderate photos [Not built]

Proposed for v1.x. See Risks.

5.9 Team challenges (CHAL)

CHAL-01 Create challenge [Built]

A captain challenges another team, picking the day and time, the place (required) and the rules, with an optional message.

- AC1: Given no team selected, it shows "Pick a team to challenge." Given no location, it shows "Pick where you want to play."
- AC2: The rules allow the challenge only if the creator is captain of `fromTeamId`, and `toCaptainId` matches the target team's captain.

CHAL-02 Respond to challenge [Built]

The challenged captain sees incoming challenges on Home and accepts or declines.

- AC1: When the challenge is accepted, then status becomes `accepted`, a scheduled match is created with both rosters, the challenge's time, location and rules, and `challengeId`, and the user is taken to the match page.
- AC2: When declined, then status becomes `declined`.
- AC3: Only the challenged captain can accept or decline, and only while the challenge is pending.

CHAL-03 Cancel challenge [Partial]

- AC1: The challenger can cancel a pending challenge from Home, and status becomes `cancelled`. **Gap:** the rules allow this, but no cancel control was found in the UI, and no push is sent on cancel.

CHAL-04 Challenge notifications [Built]

See NOTIF-02 and NOTIF-03.

5.10 Tournaments (TOUR)

TOUR-01 Create tournament [Built]

An organiser creates a tournament with a name (at least 3 characters), format (knockout or league), rules, start and end dates, location (required) and at least 2 teams. Teams can be registered teams or name-only entries.

- AC1: Validation messages: "Give the tournament a name.", "Pick a location.", "Add at least 2 teams."
- AC2: Only the organiser can update or delete the tournament.

TOUR-02 Knockout: seeded bracket with automatic byes [Built]

- AC1: Given 6 teams, then an 8-slot bracket is generated in standard seed order (1v8, 4v5, 2v7, 3v6), and the top 2 seeds get byes that advance automatically.
- AC2: When a fixture's match finishes, then the winner moves into the next-round slot. Rounds are named (for example Semi-finals, Final).

TOUR-03 League: round robin with standings [Built]

- AC1: Given N teams, fixtures are generated by the circle method. With an odd N, one team rests each round. Home and away alternate.

- AC2: Table points: 3 for a 3–0 or 3–1 (2–0) win, 2 for a 3–2 (2–1) win, 1 for a 2–3 (1–2) loss, 0 otherwise.
- AC3: Sort order: points, then wins, then set ratio, then point ratio, then name.

TOUR-04 Start fixtures and auto-update results [Built]

- AC1: When the organiser taps a ready fixture, then a live match is created linked by `tournamentId`/`fixtureId`, with the organiser as scorer (who can hand over scoring).
- AC2: When linked matches finish, then the bracket or table recalculates for all viewers.

TOUR-05 Champion detection [Built]

- AC1: When the final, or the last league fixture, finishes, then "🏆 Champion: {team}" shows and the organiser's client sets status to `finished`.

TOUR-06 Tournament notifications [Not built]

Proposed: notify team members when their fixture goes live. This is partly covered by NOTIF-04 for registered teams.

5.11 Location, map and nearby games (LOC, MAP)

LOC-01 Location on every match [Partial]

Every match, challenge and tournament must have a location (name plus lat/lng), picked with the Location Picker.

- AC1: Challenges and tournaments cannot be created without a location. *(Built)*
- AC2: A match cannot be created without a location. **Gap:** the match creation screen doesn't require one (`location` may be `null`), and matches without a location don't appear on the map.
- AC3: Matches created from an accepted challenge inherit the challenge's location. *(Built)*

MAP-01 Nearby map and list [Partial]

The Map tab shows a map (Google Maps on Android, Apple Maps on iOS, a simple preview on web) and a list of nearby live or upcoming games, unfinished tournaments and courts, with distance.

- AC1: Radius filters of 5 km, 25 km, 100 km and Anywhere. Type filters: All, Games, Tournaments, Courts.
- AC2: When location permission is granted, the map centres on the user ("Your location"). Otherwise it uses a default area ("Default area").
- AC3: Results are sorted by distance and each item shows a distance label (for example "1.1 km").
- **Gap (scale):** filtering happens on the client over at most 200 live or scheduled matches and 300 courts. Proposed: geohash queries before public launch.

MAP-02 Add a court [Built]

- AC1: A user adds a court with a name, surface (indoor, beach, grass, outdoor) and location. Only the user who added it can edit or delete it.

MAP-03 Ask to join a game ("I want to play") [Partial]

- AC1: On a scheduled match, when a user taps "I want to play", they are added to `interestedIds`, the button reads "I'm in ✓", and the "Want to play (n)" list shows their avatar. Tapping again removes them. *(Built. The rules only let users toggle themselves.)*
- AC2: The organiser or scorer is notified of new join requests and can accept them, which adds the player to a roster. **Gap:** there is no notification and no accept/decline step; it's an interest list only.

5.12 Notifications (NOTIF)

See §9 for the full matrix.

---

6. Key user flows

F1 – Onboarding. Install → Login (mobile number, +91 default) → reCAPTCHA → OTP screen → enter code → Create profile (name, photo, position, city) → Home.

F2 – Build a team. Profile → New team (name, colour) → Team page → Add a player by mobile number → *registered:* "Invite sent to {name}" (number shown masked under "Invited") / *unregistered:* refused with the register-first message → the invitee sees "{Team} wants you on the team" on Home (plus a push) → Join team or Decline → the roster updates (starters, subs).

F3 – Score a quick match. Score tab / New match → pick or type Team A and Team B → rules preset → first serve → location → Start now → split-screen scoring → tap rally winners → optional player credit → timeouts, side-switch prompts, undo → set and match won automatically → full time → share.

F4 – Follow live. Home "Live now" or Map → match page → real-time scoreboard, stats, timeline, photos.

F5 – Challenge. Home → Challenge → choose opponent, date and time, place, rules, message → the opponent captain gets a push → Accept (a scheduled match is created) or Decline (the challenger gets a push either way).

F6 – Tournament. Tournaments tab → Create tournament → format, rules, dates, location, teams → fixtures generated → organiser starts a fixture (becomes scorer, may hand over) → results feed the bracket or table → champion shown.

F7 – Traveller finds a game. Map tab → allow location → pick a radius → browse games, tournaments and courts → open a scheduled match → "I want to play" → Directions.

F8 – Offline scoring. Scoring in a dead zone → the indicator shows "Offline – saved on phone" → connectivity returns → "Synced" → followers catch up.

---

7. Data model sketch (Cloud Firestore and Storage)

Collection / path	Key fields	Notes
`users/{uid}`	`name`, `phone` (E.164), `photoURL`, `position`, `city`, `createdAt`	Owner writes only; `phone` must equal the token's phone. Readable by signed-in users (see PROF-04 gap).
`phoneIndex/{e164}`	`uid`	Registration check. `get` only, never `list`. Self-claim only.
`pushTokens/{uid}`	`token` (Expo push token)	Private to the owner. Read by Cloud Functions with admin access.

Collection / path	Key fields	Notes
`teams/{teamId}`	`name`, `nameLower`, `color`, `captainId`, `memberIds[]`, `subIds[]`, `pendingPhones[]`, `city`, `createdAt`	`pendingPhones` holds registered numbers only. Prefix search on `nameLower`.
`matches/{matchId}`	`title`, `teamA`/`teamB` {`teamId`, `name`, `color`, `players[]`}, `rules`, `firstServe`, `events[]`, `summary`, `status` (scheduled/live/finished/abandoned), `createdBy`, `scorerId`, `participantIds[]`, `location`, `scheduledAt`, `startedAt`, `finishedAt`, `tournamentId`, `fixtureId`, `challengeId`, `stream` (reserved), `interestedIds[]`, `createdAt`, `updatedAt`	The event log is embedded. State is derived by replaying it.
`matches/{matchId}/photos/{photoId}`	`url`, `storagePath`, `uploadedBy`, `createdAt`	Created by participants only.
`challenges/{id}`	`fromTeamId/Name`, `fromCaptainId`, `toTeamId/Name`, `toCaptainId`, `when`, `location`, `message`, `rules`, `status` (pending/accepted/declined/cancelled), `matchId`	Cannot be deleted.
`tournaments/{id}`	`name`, `organizerId`, `format` (knockout/league), `rules`, `startDate`, `endDate`, `location`, `teams[]`, `fixtures[]` {`id`, `round`, `index`, `homeId`, `awayId`, `homeFrom`, `awayFrom`, `matchId`}, `status` (registration/running/finished)	Results are derived from linked matches.
`courts/{id}`	`name`, `location`, `surface`, `addedBy`, `createdAt`	Community-contributed.
Storage `avatars/{uid}/…`	image under 10 MB	Owner writes only.
Storage `matches/{matchId}/{uid}/…`	image under 10 MB	Match participants only, under their own uid.

Event model: `PointEvent {side, playerId?, kind? (kill|block|ace|tip), at}` and `TimeoutEvent {side, at}`. `MatchSummary` is a compact snapshot (sets, current score, winner) stored next to the log for list screens. Note: a *kill* is labelled "Spike" in the UI.

Scale note: the event log is embedded in the match document. A best-of-5 match has roughly 200–250 events, well within Firestore's 1 MiB document limit. Proposed: monitor document size if per-event metadata grows.

---

8. Permissions and roles

Action	Anonymous	Signed-in user	Team member	Captain	Match participant	Scorer	Match creator	Organiser
View matches, teams, tournaments, courts, profiles	✗	✓	✓	✓	✓	✓	✓	✓
Edit own profile / claim own number	✗	✓						
Create team	✗	✓						

Action	Anonymous	Signed-in user	Team member	Captain	Match participant	Scorer	Match creator	Organiser
Edit team, invite registered numbers, remove members, cancel invites				✓				
Accept or decline own invite		✓ (if invited)						
Leave team			✓ (not captain)					
Create match	✗	✓ (must be a participant)						
Change score / status						✓		
Hand over scoring						✓	✓	
Delete match (not while live; rules only, no UI found)							✓	
Toggle self "I want to play"		✓						
Add match photos					✓			
Delete own photos					✓ (own)			
Create challenge / cancel own				✓ (from own team)				
Accept or decline challenge				✓ (challenged team)				
Create, edit or delete tournament, start fixtures		✓ (create)						✓
Add court / edit own court		✓						

All access requires a signed-in user. The server-side Firestore and Storage rules are the source of truth. The client UI mirrors them.

---

9. Notifications

Delivered through the Expo Push API (FCM on Android, APNs on iOS) from Cloud Functions using `pushTokens/{uid}`. This requires the Firebase Blaze plan, `EAS_PROJECT_ID`, and a development or store build (pushes do not work in Expo Go on Android). Tapping a notification deep-links to the relevant screen. Without push, invites and challenges still appear in real time on Home.

ID	Trigger	Recipient	Message	Deep link	Status
NOTIF-01	Registered number added to `pendingPhones`	Invitee	"🏐 Team invite – You've been invited to join {team}."	Team	**[Built]** `onTeamInvite`
NOTIF-02	Challenge created	Challenged captain	"⚡ New challenge! {from} challenged {to}..."	Challenge/Home	**[Built]** `onChallengeCreated`
NOTIF-03	Challenge accepted or declined	Challenger captain	"✅ Challenge accepted" / "Challenge declined"	Match / Home	**[Built]** `onChallengeUpdated`
NOTIF-04	Match created live, or status changes to live	Participants except the scorer	"🔴 Your match is live – {A} vs {B}..."	Match	**[Built]** `onMatchCreatedLive`, `onMatchLive`
NOTIF-05	Challenge cancelled	Challenged captain	proposed	Home	**[Not built]**
NOTIF-06	New "I want to play" request	Match creator / scorer	proposed	Match	**[Not built]**
NOTIF-07	Invite accepted or declined	Captain	proposed	Team	**[Not built]**
NOTIF-08	Match finished (result)	Participants / followers	proposed	Match	**[Not built]**
NOTIF-09	Upcoming scheduled match reminder	Participants	proposed	Match	**[Not built]**

Acceptance criteria for NOTIF-01 to NOTIF-04:

- AC1: Given a recipient with a stored push token on a store or dev build, when the trigger happens, then a push arrives. Proposed target: p95 ≤ 10 s.
- AC2: Tapping the push opens the deep-linked screen.
- AC3: A user without a token gets no push and nothing errors. A failed send is logged with `logger.warn`.
- AC4: The OS notification permission is requested at a sensible point, and denial doesn't block any feature.

---

10. Non-functional requirements

ID	Category	Requirement	Status
NFR-01	Offline	Scoring must keep working with no connection, with no data loss across app restarts. Unsynced points sync automatically on reconnect (see OFF-01 to OFF-03).	Partial (OFF-02 AC3)
NFR-02	Real-time latency	*Proposed:* scorer tap to follower display p95 ≤ 2 s and p99 ≤ 5 s on 4G.	Not measured
NFR-03	Scoring UI responsiveness	*Proposed:* local score update ≤ 100 ms after a tap on a mid-range Android device.	Not measured

ID	Category	Requirement	Status
NFR-04	Security rules	All writes are validated by Firestore and Storage rules: only the scorer changes the score; captains invite only registered numbers; self-only invite responses; private push tokens; images only, under 10 MB. The rules test suite (22 tests, Firebase emulator) must pass before deploying.	Built (gaps: TEAM-05, TEAM-02 AC4)
NFR-05	Score payload validation	*Proposed:* the rules validate the shape of `events` and `summary` and allowed `status` transitions.	Not built
NFR-06	Phone number privacy	Full numbers are never shown to other users. `phoneIndex` can't be listed. Numbers are not readable from other users' profile documents.	Partial (PROF-04)
NFR-07	Location privacy	Device location is used only on the device to centre the map and calculate distances, and is never stored or uploaded. Only locations the user explicitly picks for matches, challenges, tournaments and courts are stored. Location permission is optional, with a fallback to a default area.	Built (per code review: `getCurrentLocation` is used only by the Map screen to centre the view and by `LocationPicker` to pre-centre or suggest a pin; only the place the user confirms is saved)
NFR-08	Accessibility	*Proposed:* all interactive controls have accessibility labels and roles; touch targets ≥ 44×44 pt; WCAG AA colour contrast for text; score halves announce "{team} {points}"; supports OS font scaling without clipping; haptics are not the only feedback.	Partial (large tap targets and haptics built; only 2 `accessibilityLabel/Role` usages found)
NFR-09	Performance	*Proposed:* cold start ≤ 3 s on a mid-range device; Home first content ≤ 2 s on 4G; match page with 250 events renders in ≤ 500 ms.	Not measured
NFR-10	Scalability	*Proposed:* geohash-based nearby queries and server-side stat aggregates before public launch (current limits: 50 live/upcoming on Home, 200 matches on the map, 300 courts, 200 matches per user history).	Partial
NFR-11	Reliability	*Proposed:* crash-free sessions ≥ 99.5%. Crash reporting (for example Sentry or Crashlytics) needs to be integrated.	Not built
NFR-12	Data retention and deletion	Users can delete their account and data in the app (required by Apple App Store and Google Play policies).	Not built (`users` delete is denied by the rules)
NFR-13	Secrets	Firebase web config comes from env vars; `.env` is git-ignored; no secrets in the repo. `Match.stream.ingestUrl` must never be shown to followers.	Built
NFR-14	Platform support	Android and iOS via EAS Build; Expo Go supported for development (except Android push).	Built
NFR-15	Test coverage	Pure-logic unit tests (scoring, rules, stats, tournaments, phone, geo) and rules tests must pass in CI.	Built: 51 Jest tests passing at review time; 22 rules tests present (not run in this review because the emulator needs Java). CI not configured.

11. Analytics and success metrics (all proposed)

No analytics SDK is integrated (none found in code). The events and targets below are **proposed** and need validating after launch.

11.1 Proposed event taxonomy

`auth_otp_requested`, `auth_otp_verified`, `profile_created`, `team_created`, `team_invite_sent`, `team_invite_refused_unregistered`, `team_invite_accepted/declined`, `match_created` (preset, has_location, scheduled/now), `match_started`, `point_scored` (sampled), `point_credited`, `undo_used`, `match_finished`, `match_abandoned`, `offline_session` (duration, points queued), `sync_completed` (latency), `match_viewed_live`, `photo_uploaded`, `challenge_sent/accepted/declined`, `tournament_created` (format, teams), `fixture_started`, `map_opened` (radius), `want_to_play_toggled`, `push_opened` (type).

11.2 Proposed success metrics

Metric	Definition	Proposed v1 target (90 days after launch)
Activation	% of verified users who create a profile and either join a team or follow or score a match within 7 days	≥ 40%
Scored matches per week	Matches reaching `finished`	Growth week over week; baseline to be set
Scoring completion rate	finished ÷ (finished + abandoned + stale live)	≥ 85%
Offline integrity	Matches with queued offline points that sync fully	≥ 99.9%
Live followers per match	Unique viewers of a live match page	≥ 3 median
Credit adoption	% of points with player credit	Track only (no target)
Invite acceptance	Accepted ÷ sent invites	≥ 60%
Unregistered-refusal recovery	% of refused numbers that register and are added within 7 days	Track (informs the no-invite-link decision)
Challenge acceptance	Accepted ÷ sent	≥ 50%
Map engagement	% of WAU opening the Map; "I want to play" taps per week	Track
D30 retention	Users active on day 30	≥ 25%
Crash-free sessions		≥ 99.5%

12. Release milestones (proposed)

Milestone	Content	Exit criteria
M0 – Current build (done)	All v1 features at Built or Partial status (Appendix A); Demo mode; rules and unit tests	51 unit tests passing; README setup verified
M1 – v1 gap closure	Fix TEAM-05 (captain cannot add members directly), TEAM-02 AC3/AC4 (auto-sub on accept, server-side caps), LOC-01 (location required on matches), PROF-04 (hide phone from profile docs), OFF-02 AC3 (background sync), PHOTO-03 (delete own photo)	All requirements tagged Partial for v1 are Built; rules tests extended for each fix
M2 – Compliance and quality	Account deletion (NFR-12), photo reporting, accessibility pass (NFR-08), analytics (§11), crash reporting (NFR-11), privacy policy, store listings	Accessibility checklist passed; analytics events firing; store review checklist complete

Milestone	Content	Exit criteria
M3 – Closed beta	EAS preview builds for 3–5 local clubs or communities (proposed); push enabled (Blaze plan)	Proposed: ≥ 50 matches scored end to end, no data-loss incidents, p95 latency measured
M4 – Public v1 launch	Play Store and App Store release	Proposed: crash-free ≥ 99.5%, latency targets met, no open P0/P1 bugs
v1.x / v2	Geohash nearby queries, join-request approval and notifications, more notifications (NOTIF-05 to NOTIF-09), captain transfer, lineups and substitutions, **live video streaming**	Prioritised from beta feedback

13. Risks

#	Risk	Impact	Mitigation (proposed)
R1	Captain can add arbitrary uids to `memberIds` by writing to Firestore directly (TEAM-05 gap)	Breaks the "registered + accepted only" promise; could tag users without consent	Restrict captain writes to removals; add rules tests
R2	Phone numbers readable from `users/{uid}` by any signed-in user	Privacy breach; scraping by uid	Move `phone` to a private doc; keep `phoneIndex` get-only
R3	SMS OTP cost and abuse (Blaze billing, SMS pumping)	Cost spikes	Firebase App Check, SMS region policy, rate limits, test numbers in development
R4	Registered-only team rule slows team formation (no invite links)	Lower activation	Measure the refusal-recovery metric; clear in-app guidance to "ask them to register"
R5	Client-side nearby filtering doesn't scale past ~200 matches	Missing results, slow map	Geohash queries (M1/M2)
R6	No account deletion	App store rejection	Build NFR-12 before submission
R7	User-generated photos unmoderated	Abuse, store policy issues	Report/hide flow; participant-only uploads already limit exposure
R8	Scorer integrity: one scorer can enter wrong scores	Disputed results, especially in tournaments	Undo, handover, visible timeline; consider a post-match confirmation by both captains (open question)
R9	Push depends on Blaze plan, EAS project and dev builds	No notifications in some environments	Real-time Home lists as a fallback; deployment checklist
R10	Demo mode accidentally shipped	Fake data in production	Build-time assertion that Firebase config is present for production profiles
R11	Firestore in-memory cache only; retries only while the scoring screen is open	Delayed sync if the scorer leaves the screen	Background sync job on app start; consider persistent cache
R12	Embedded event log grows with metadata	Hitting document size or write-contention limits	Monitor; split events into a subcollection if needed

14. Open questions

1. **Guest players in quick matches.** Match creation allows typed-in team and player names (guests), while teams require registered players. Is that intended for v1, or should quick matches also be limited to registered users? Guest points can't feed career stats.

2. **"I want to play" outcome.** Should the organiser accept or decline join requests (adding the player to a roster) and be notified, or is an interest list enough for v1?
3. **Location mandatory on matches.** Should quick matches started with "Start now" require a location (strict v1 reading), or allow an optional location with a nudge?
4. **Match and location visibility.** All matches, including scheduled ones with locations, are visible to every signed-in user. Do we need private matches or teams, or a "hide exact location" option?
5. **Result disputes.** Should both captains confirm a finished match before it counts toward tournaments and career stats?
6. **Tournament teams.** Should tournaments allow name-only teams, or require registered teams so that stats and notifications work?
7. **Sub assignment on accept.** When a 7th player accepts, should they always become a sub (as in Demo mode), or should the captain decide?
8. **Captain lifecycle.** Captain transfer, co-captains, and what happens if the captain wants to leave or delete the team.
9. **Markets and phone formats.** India-only at launch (+91 default), or a multi-country picker in v1?
10. **Account deletion semantics.** On deletion, are past match records anonymised ("Former player") or removed?
11. **Analytics and crash tooling choice** (Firebase Analytics/Crashlytics vs. alternatives), and the matching consent and privacy-policy wording.
12. **Live video (v2).** Preferred provider and model (RTMP ingest vs. WebRTC vs. YouTube link) for planning, given that `Match.stream` is already reserved.

Appendix A – Implementation status mapped to code

Based on a code review of `/workspace/volleylive` on 10 Oct 2026, plus README and screenshots `01`–`16` in `/workspace/volleylive-screens`. Unit tests were run: **4 suites, 51 tests, all passing**. Security-rules tests (22 in `rules-tests/rules.test.js`) were not run, because they need the Firebase emulator and Java.

Req ID	Feature	Status	Code evidence
AUTH-01	Phone OTP sign-in	Built	`src/app/login.tsx`, `src/services/backend/firebaseBackend.ts` (`sendOtp`), `src/core/phone.ts` (`normalizePhone`); screens 01, 02
AUTH-02	reCAPTCHA in Expo	Built	`src/components/PhoneVerifier.tsx`, `PhoneVerifier.web.tsx`
AUTH-03	Demo mode	Built	`src/services/backend/demoBackend.ts`, `demoSeed.ts`, `src/services/config.ts`
AUTH-04	Sign out	Built	`src/app/(tabs)/profile.tsx`
PROF-01	Profile setup	Built	`src/app/setup-profile.tsx`, `src/components/ProfileForm.tsx`, `firestore.rules` (users); screen 03
PROF-02	Phone index	Built	`firestore.rules` (`phoneIndex`), rules tests "cannot claim another number", "phone lookup is get-only"
PROF-03	Edit profile / avatar	Built	`src/app/edit-profile.tsx`, `storage.rules` (`avatars`)
PROF-04	Phone privacy	Partial	`maskPhone` in `src/app/team/[id].tsx`; `users/{uid}.phone` readable by all signed-in users
TEAM-01	Create team	Built	`src/app/team/new.tsx`, `firestore.rules` (teams create)
TEAM-02	6 + 6 roster	Partial	`TEAM_SIZE`/`MAX_SUBS` in `src/core/types.ts`; client checks in `team/[id].tsx`; Firebase `acceptTeamInvite` doesn't set `subIds`; no combined cap in the rules

Req ID	Feature	Status	Code evidence
TEAM-03	Registered-only add	Built	`team/[id].tsx` (`findUserByPhone`, refusal message), `invitesOnlyRegistered()` rule; screens 15, 16
TEAM-04	Accept / decline invite	Built	`src/app/(tabs)/index.tsx` (Team invites), `acceptingInvite()`/`decliningInvite()` rules; screen 04
TEAM-05	No invite bypass	Partial	Captain update rule doesn't restrict `memberIds` additions
TEAM-06	Remove / leave / cancel invite	Built	`team/[id].tsx` (`remove`, `leave`, cancel pending), `leaving()` rule
TEAM-07	Captain-only edit	Built	rules test "non-captain cannot edit team"
SCORE-01	Split-screen tap scoring	Built	`src/app/match/[id]/score.tsx`, `src/core/scoring.ts` (`addPoint`); screen 10
SCORE-02	Undo	Built	`scoring.ts` (`undo`), `score.tsx` (`doUndo`)
SCORE-03	Serve tracking	Built	`computeState` in `scoring.ts`; `scoring.test.ts`
SCORE-04	Set/match point and wins	Built	`scoring.ts` (`setPoint`, `matchPoint`, `finished`)
SCORE-05	Indoor/Beach/Custom rules	Built	`src/core/rules.ts`, `src/components/RulesEditor.tsx`
SCORE-06	Timeouts	Built	`callTimeout` in `scoring.ts`; `score.tsx`
SCORE-07	Side switches / swap	Built	`sideSwitchDue` in `scoring.ts`; "Switch sides" toast, Swap button in `score.tsx`
SCORE-08	Player point credit	Built	`attributeLastPoint`, `POINT_KINDS` (kill/block/ace/tip); screen 09
SCORE-09	Match setup	Partial	`src/app/match/new.tsx`, `src/services/matchFactory.ts`; location optional; screen 08
SCORE-10	Abandon	Built	`score.tsx` (`status: 'abandoned'`)
SCORE-11	Share result	Built	`shareText` in `src/core/stats.ts`; `match/[id]/index.tsx`
ROLE-01	Single scorer	Built	`firestore.rules` (`isScorer`), `score.tsx` guard; rules tests
ROLE-02	Hand over scoring	Built	`match/[id]/index.tsx` (scorer chips), rules handover clause
ROLE-03	Read-only followers	Built	`match/[id]/index.tsx`; screen 05
OFF-01	Local-first scoring	Built	`score.tsx` (AsyncStorage `score-cache-{id}`, sync indicator)
OFF-02	Retry / recovery	Partial	5 s retry while the screen is mounted; restore on reopen; default in-memory Firestore cache (`src/services/firebase.ts`)
OFF-03	Conflict model	Built	Event-sourced engine; `pushSeq` in `score.tsx`
OFF-04	Followers offline	Not built (by design)	README §7
LIVE-01	Real-time scoreboard	Built	`watchMatch` (`onSnapshot`) in `firebaseBackend.ts`; screen 05
LIVE-02	Home live / upcoming	Built	`(tabs)/index.tsx`, `watchLiveMatches` (limit 50); screen 04
LIVE-03	Stats / timeline tabs	Built	`match/[id]/index.tsx`, `timeline()`; screen 06
LIVE-04	Live video	Deferred	`StreamInfo` in `types.ts`; placeholder in `match/[id]/index.tsx`
STAT-01	Per-match stats	Built	`src/core/stats.ts` (`playerStats`, `teamTotals`, `longestRun`); `stats.test.ts`
STAT-02	Career totals	Built	`careerStats` in `stats.ts`; `profile.tsx`; screen 14
STAT-03	Match history	Built	`watchMyMatches` (limit 200); `profile.tsx`
PHOTO-01	Add photos	Built	`uploadMatchPhoto`; `storage.rules`; photos rules
PHOTO-02	View album	Built	`watchPhotos`; Photos tab
PHOTO-03	Delete own photo	Partial	Allowed by the rules; no UI or backend method

Req ID	Feature	Status	Code evidence
PHOTO-04	Report / moderate	Not built	n/a
CHAL-01	Create challenge	Built	`src/app/challenge/new.tsx`; challenges rules
CHAL-02	Respond to challenge	Built	`(tabs)/index.tsx` (`respond`, `acceptChallenge`)
CHAL-03	Cancel challenge	Partial	Allowed by the rules; no cancel control found in `src/app`; no push
CHAL-04	Challenge pushes	Built	`functions/src/index.ts`
TOUR-01	Create tournament	Built	`src/app/tournament/new.tsx`
TOUR-02	Knockout bracket	Built	`generateKnockout`, `seedOrder`, `resolveKnockout` in `src/core/tournament.ts`; screen 13
TOUR-03	League table	Built	`generateRoundRobin`, `matchTablePoints`, `standings`; `tournament.test.ts`
TOUR-04	Start fixtures / auto results	Built	`src/app/tournament/[id].tsx`, `src/core/results.ts`
TOUR-05	Champion detection	Built	`leagueChampion`, `championId`; `tournament/[id].tsx`
TOUR-06	Tournament pushes	Not built	n/a (NOTIF-04 covers registered participants)
LOC-01	Location on every match	Partial	`LocationPicker` required for challenges, tournaments and courts; optional in `match/new.tsx`
MAP-01	Nearby map	Partial	`src/app/(tabs)/map.tsx`, `src/core/geo.ts` (`nearby`, `distanceKm`), `AppMap.tsx`/`.web.tsx`; client-side filtering; screen 11
MAP-02	Add court	Built	`src/app/court/new.tsx`; courts rules
MAP-03	"I want to play"	Partial	`interestedIds` toggle and rule; no approval or notification
NOTIF-01 to NOTIF-04	Pushes	Built	`functions/src/index.ts`, `src/services/notifications.ts` (token, routing)
NOTIF-05 to NOTIF-09	Extra pushes	Not built	n/a
NFR-04	Security rules	Built (with gaps)	`firestore.rules`, `storage.rules`, `rules-tests/`
NFR-08	Accessibility	Partial	2 accessibility props found across `src/`
NFR-11	Crash reporting	Not built	n/a
NFR-12	Account deletion	Not built	`users` delete is `false` in the rules
§11	Analytics	Not built	No analytics SDK or `logEvent` found

A.1 Summary: built vs missing

- **Built:** OTP auth (with Demo mode), profiles, team creation, registered-only invites with accept/decline, captain controls, the full scoring engine (tap scoring, undo, serve, set and match logic, Indoor/Beach/Custom rules, timeouts, side switches, player credit), single-scorer enforcement and handover, local-first offline scoring, real-time scoreboard, match stats, timeline, career totals and history, photo albums, challenges with pushes, knockout and league tournaments with brackets, tables and champion, nearby map with radius and type filters, courts, the "I want to play" interest list, and pushes for invites, challenges and match-live.
- **Partial:** phone privacy at the data layer; 6+6 enforcement (auto-sub on accept, server-side caps); membership-bypass rule gap; required location on matches; background offline sync; photo delete; challenge cancel (no UI, no push); join-request approval; map scalability; accessibility.

- **Not built:** analytics, crash reporting, account deletion, photo moderation, extra notifications (NOTIF-05 to NOTIF-09, tournament pushes), score payload validation in the rules.
- **Deferred:** live video streaming.